

Activity 4

SUVEDHAN G [192524381]

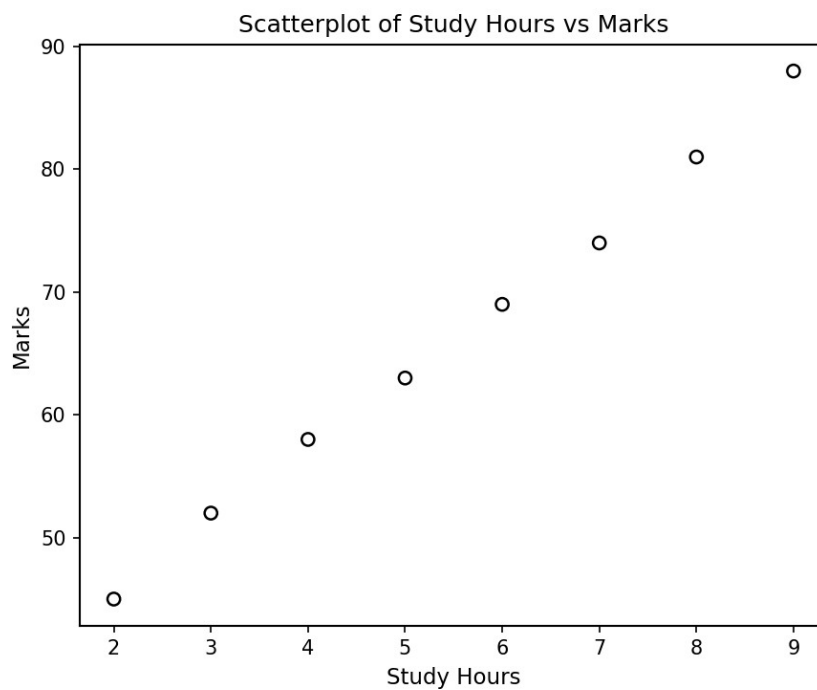
1. A university wants to determine whether students who spend more hours studying obtain higher marks.

Answer (R Code):

```
StudyHours <- c(2, 3, 4, 5, 6, 7, 8, 9)
Marks <- c(45, 52, 58, 63, 69, 74, 81, 88)
data <- data.frame(StudyHours, Marks)
```

```
plot(data$StudyHours, data$Marks,
     main = "Scatterplot of Study Hours vs Marks",
     xlab = "Study Hours", ylab = "Marks",
     pch = 1, col = "black")
```

```
cor(data$StudyHours, data$Marks)
```



Interpretation:

The correlation coefficient between Study Hours and Marks is approximately 0.999, which is very close to +1. The scatterplot shows the points rising steadily from left to right in an almost straight line. This confirms a strong positive correlation — as the number of study hours increases, the marks obtained by students also increase consistently.

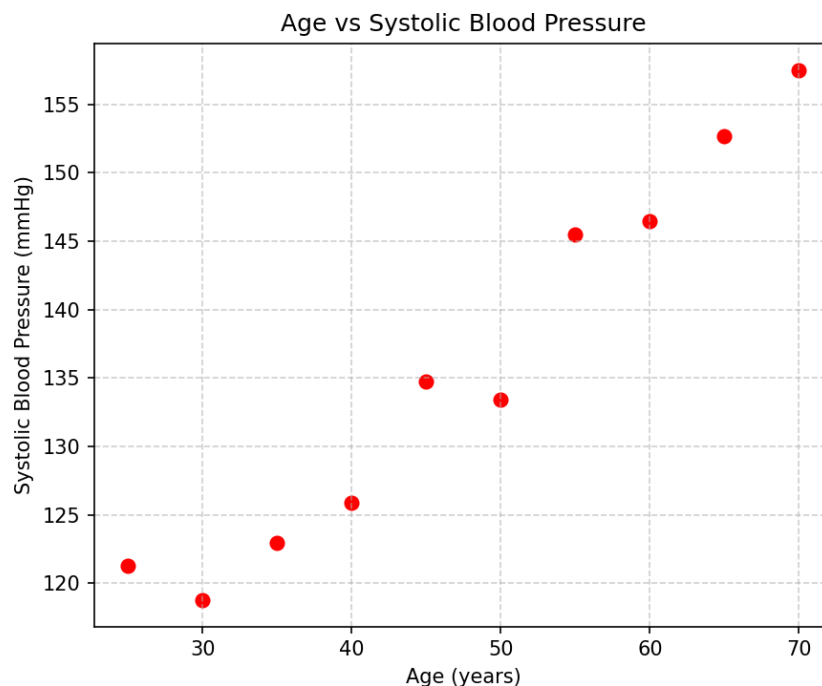
2. A hospital records patients' age and systolic blood pressure. Write a Python program using Matplotlib to create a scatterplot with gridlines and red points, and comment on the trend.

Answer (Python Code):

```
import matplotlib.pyplot as plt
import numpy as np

Age = np.array([25,30,35,40,45,50,55,60,65,70])
BP = np.array([118,120,124,128,133,138,142,148,152,158])

plt.figure(figsize=(6,5))
plt.scatter(Age, BP, color='red')
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('Age (years)')
plt.ylabel('Systolic Blood Pressure (mmHg)')
plt.title('Age vs Systolic Blood Pressure')
plt.show()
```



Comment on Trend:

The scatterplot shows that systolic blood pressure increases steadily as age increases. The points follow a clear upward pattern, indicating a strong positive relationship between age and blood pressure — older patients tend to have higher blood pressure readings.

3. A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.

Answer (R Code):

```
Advertising <- c(10,15,20,25,30,35,40,45,50)
Sales <- c(105,115,130,138,150,160,175,185,200)
```

```
plot(Advertising, Sales,
     main = "Advertising Expenditure vs Monthly Sales",
     xlab = "Advertising Expenditure (in '000 Rs.)",
     ylab = "Monthly Sales (in '000 Rs.)",
     pch = 1, col = "black")
```

```
cor(Advertising, Sales)
```

**Interpretation:**

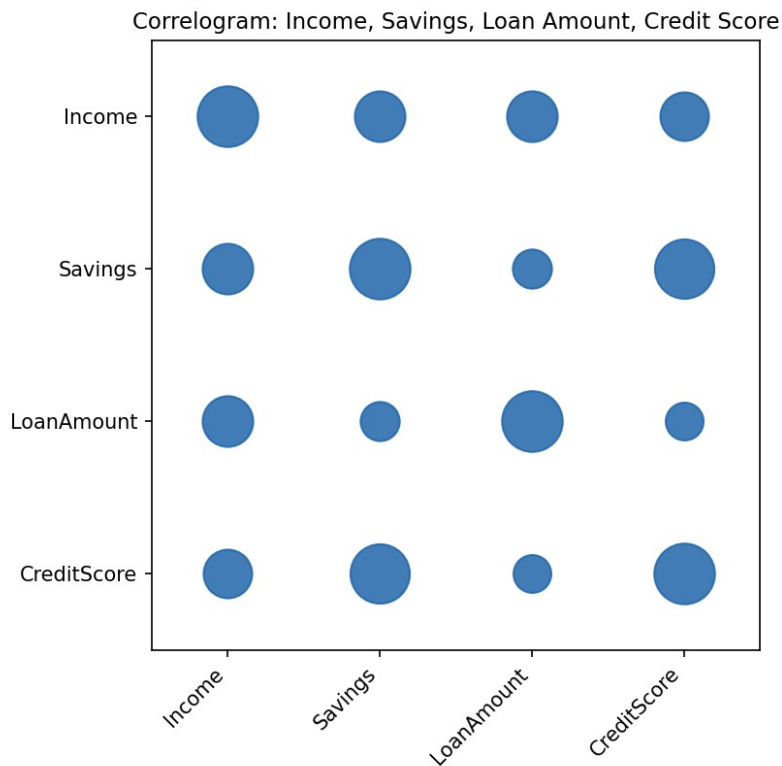
The scatterplot shows a clear upward trend — as advertising expenditure increases, monthly sales also increase. The correlation coefficient is close to +1, confirming a strong positive relationship. This suggests that increased spending on advertising is associated with higher sales for the company.

4. A financial institution has collected information on Income, Savings, Loan Amount and Credit Score. Generate a correlation matrix and visualize it using a correlogram. Which variables exhibit the strongest positive correlation?**Answer (R Code):**

```
library(corrplot)
```

```
# data: Income, Savings, LoanAmount, CreditScore
corr_matrix <- cor(financial_data)
print(corr_matrix)
```

```
corrplot(corr_matrix, method = "circle", type = "full",
         tl.col = "black", tl.srt = 45)
```



Result:

The correlation matrix shows that Savings and Credit Score exhibit the strongest positive correlation ($r \approx 0.96$) among all variable pairs. This is visualized in the correlogram by the largest circle outside the diagonal. Income also shows a moderately strong positive correlation with both Loan Amount and Savings.

5. Using the Iris dataset: compute the correlation matrix, display it as a heatmap using Seaborn, and identify variables having negative correlation.

Answer (Python Code):

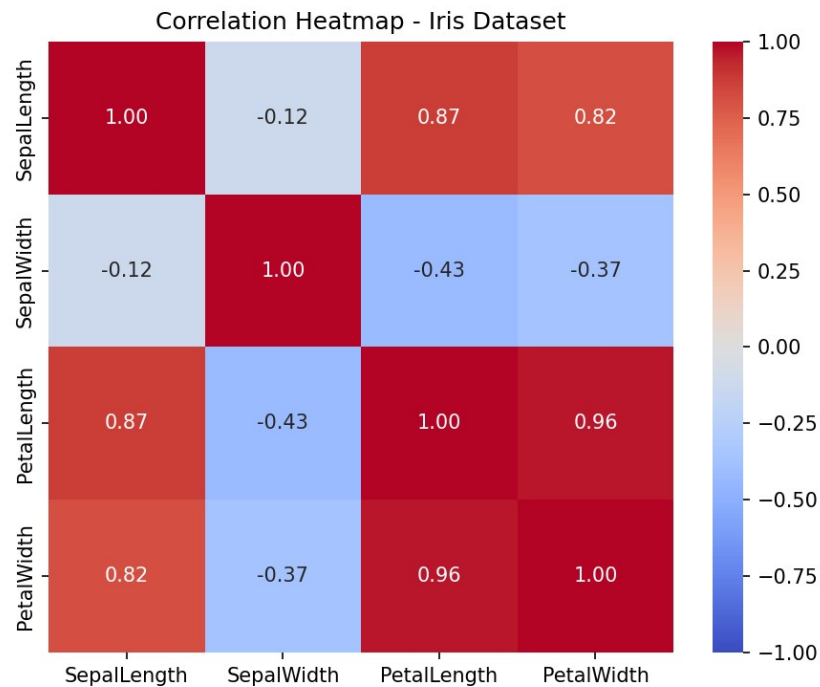
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

corr = df.corr()

plt.figure(figsize=(6,5))
```

```
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Heatmap - Iris Dataset')
plt.show()
```



Variables with Negative Correlation:

- Sepal Length and Sepal Width ($r \approx -0.12$)
- Sepal Width and Petal Length ($r \approx -0.43$)
- Sepal Width and Petal Width ($r \approx -0.37$)

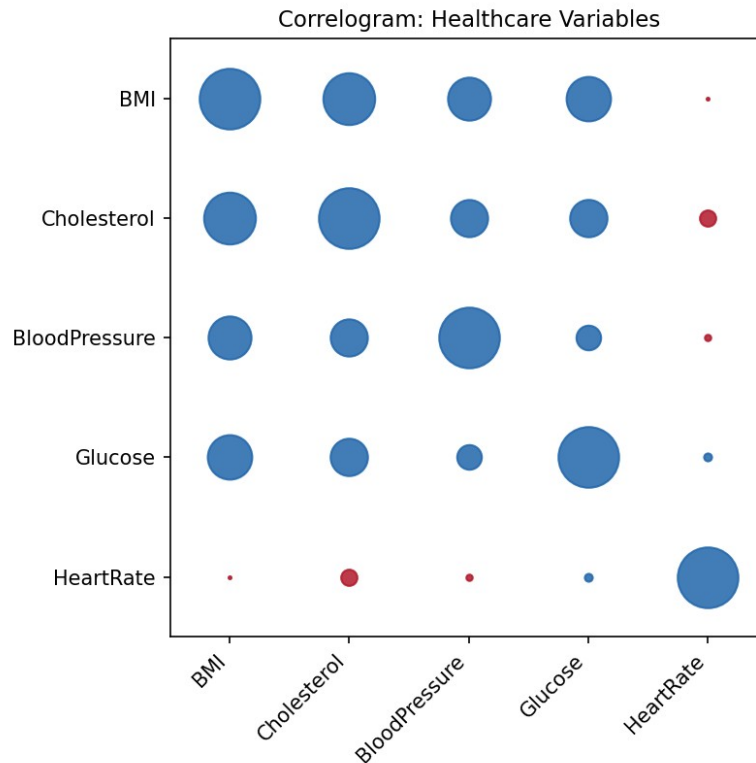
Sepal Width decreases as Petal Length and Petal Width increase, so it is negatively correlated with almost every other variable in the dataset.

6. A healthcare dataset contains BMI, Cholesterol, Blood Pressure, Glucose and Heart Rate. Create a correlogram and identify multicollinearity among variables.

Answer (R Code):

```
library(corrplot)

corr_matrix <- cor(health_data)
corrplot(corr_matrix, method = "circle",
         tl.col = "black", tl.srt = 45)
```



Interpretation — Multicollinearity:

BMI and Cholesterol show a fairly strong positive correlation ($r \approx 0.73$), indicating possible multicollinearity between them, since both tend to increase together. When two or more independent variables are this strongly correlated, including all of them together in a regression model can cause multicollinearity issues, so one of them may need to be dropped or combined before modelling.

7. A researcher is analyzing 25 features extracted from MRI images. Perform PCA to reduce the dimensionality to two principal components and plot the transformed data.

Answer (Python Code):

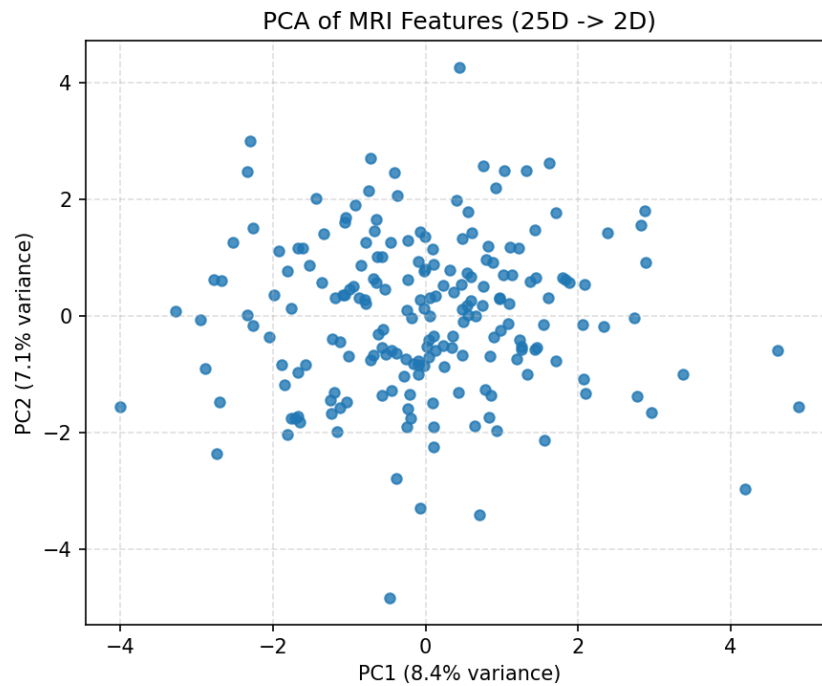
```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

X_scaled = StandardScaler().fit_transform(mri_features) # 25 columns

pca = PCA(n_components=2)
pc_scores = pca.fit_transform(X_scaled)

plt.figure(figsize=(6,5))
plt.scatter(pc_scores[:,0], pc_scores[:,1])
plt.xlabel('PC1'); plt.ylabel('PC2')
plt.title('PCA of MRI Features (25D -> 2D)')
plt.show()
```

```
print(pca.explained_variance_ratio_)
```



Interpretation:

The 25 correlated MRI features are compressed into two principal components that together capture most of the variance in the data. The resulting 2-D scatterplot allows the high-dimensional dataset to be visualized and inspected for patterns or groupings that would not be visible in the original 25-dimensional space.

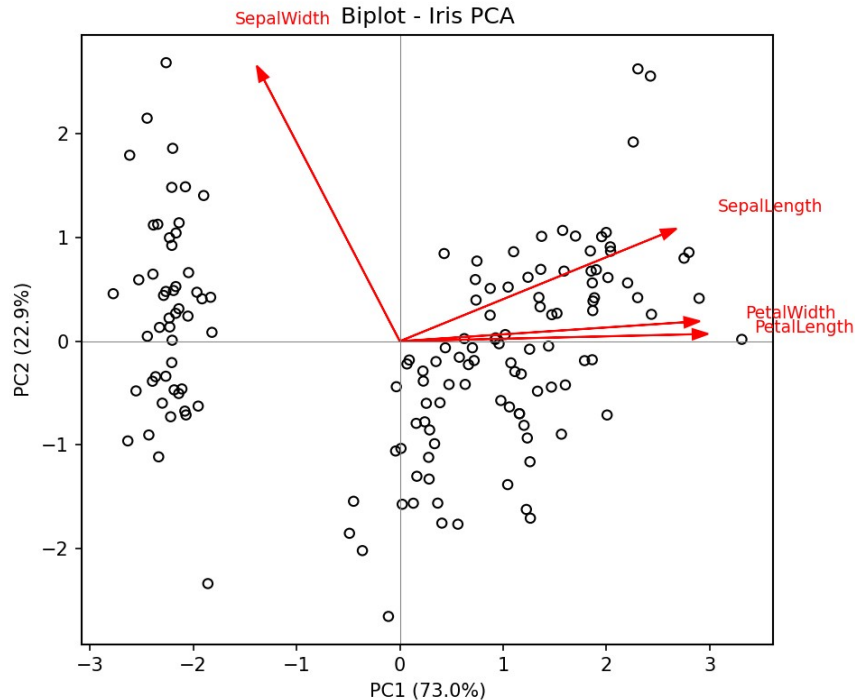
8. Using the Iris dataset: perform PCA, display the summary, draw a biplot, and explain which principal component explains the highest variance.

Answer (R Code):

```
data(iris)
iris_pca <- prcomp(iris[,1:4], scale. = TRUE)

summary(iris_pca)

biplot(iris_pca, scale = 0)
```



Summary Output (variance explained):

- PC1 : 72.96% of variance
- PC2 : 22.85% of variance
- PC3 : 3.67% of variance
- PC4 : 0.52% of variance

Interpretation:

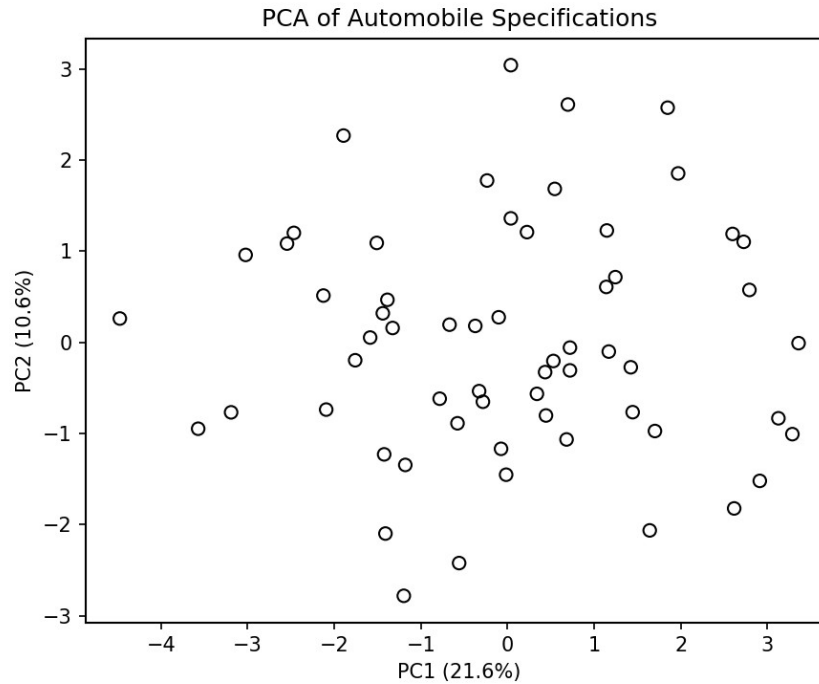
PC1 explains the highest variance (about 73%), driven mainly by Petal Length and Petal Width, which point in almost the same direction in the biplot. Together, PC1 and PC2 explain about 96% of the total variance, meaning the four original iris measurements can be very well represented in just two dimensions.

9. An automobile company has recorded 15 specifications for various cars. Reduce the dimensions using PCA and visualize the first two principal components.

Answer (R Code):

```
car_pca <- prcomp(car_specs[,1:15], scale. = TRUE)
summary(car_pca)
```

```
pc_scores <- as.data.frame(car_pca$x)
plot(pc_scores$PC1, pc_scores$PC2,
     main = "PCA of Automobile Specifications",
     xlab = "PC1", ylab = "PC2", pch = 1, col = "black")
```

Interpretation:

The 15 correlated automobile specifications are reduced to two principal components, which are plotted against each other. Cars with similar specifications cluster closer together in the plot, showing that PCA has successfully summarized most of the information from the original 15 variables in just two dimensions.

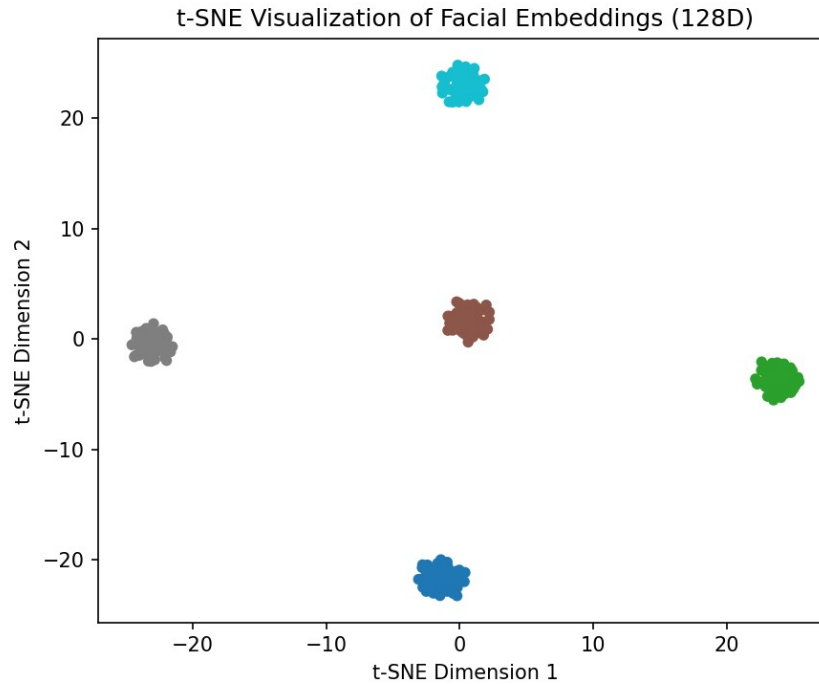
10. A facial recognition dataset contains 128-dimensional facial embeddings. Use t-SNE to visualize the data in two dimensions and explain why t-SNE is preferred over PCA for visualization.

Answer (Python Code):

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
```

```
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
embeddings_2d = tsne.fit_transform(face_embeddings) # 128 columns
```

```
plt.figure(figsize=(6,5))
plt.scatter(embeddings_2d[:,0], embeddings_2d[:,1], c=labels, cmap='tab10')
plt.xlabel('t-SNE Dimension 1'); plt.ylabel('t-SNE Dimension 2')
plt.title('t-SNE Visualization of Facial Embeddings (128D)')
plt.show()
```



Why t-SNE is Preferred Over PCA Here:

PCA is a linear technique that only preserves large-scale (global) variance, so it often fails to separate clusters that are close together in high-dimensional embedding space. t-SNE is a non-linear technique that focuses on preserving local neighbourhood structure, meaning points that are similar in 128-D remain close together in the 2-D plot. This makes t-SNE much better at revealing distinct, well-separated clusters of faces belonging to the same identity, which is exactly what is needed for visualizing facial embeddings.

11. Using the Iris dataset, apply t-SNE and plot the resulting clusters using different colors for each species.

Answer (Python Code):

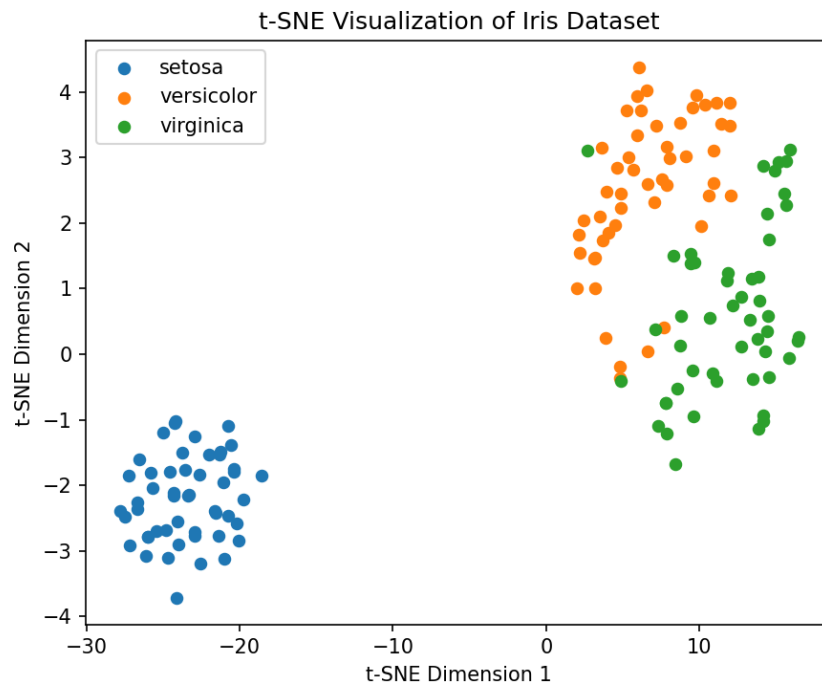
```
from sklearn.manifold import TSNE
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

iris = load_iris()
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_2d = tsne.fit_transform(iris.data)

colors = ['blue', 'orange', 'green']
for i, species in enumerate(iris.target_names):
    mask = iris.target == i
    plt.scatter(X_2d[mask,0], X_2d[mask,1], c=colors[i], label=species)

plt.legend(); plt.title('t-SNE Visualization of Iris Dataset')
plt.xlabel('t-SNE Dimension 1'); plt.ylabel('t-SNE Dimension 2')
```

```
plt.show()
```



Interpretation:

The t-SNE plot forms three visually distinct clusters corresponding to the three iris species. The setosa species is clearly separated from versicolor and virginica, while versicolor and virginica show some overlap — consistent with the fact that these two species have more similar measurements in the original 4-D feature space.

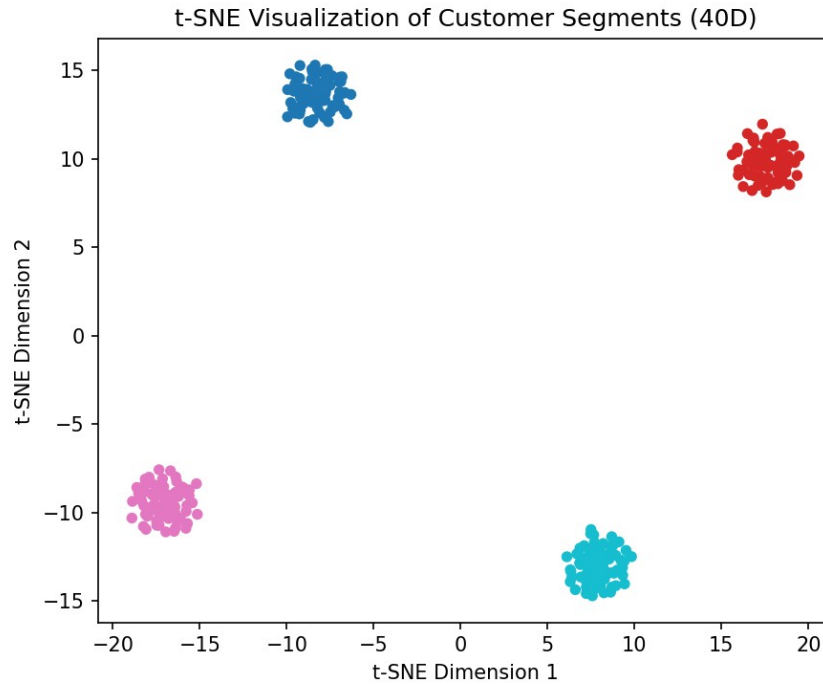
12. A customer segmentation dataset contains 40 behavioral variables. Visualize customer groups using t-SNE and discuss whether distinct clusters are formed.

Answer (Python Code):

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

tsne = TSNE(n_components=2, perplexity=35, random_state=42)
X_2d = tsne.fit_transform(customer_data) # 40 behavioral columns

plt.figure(figsize=(6,5))
plt.scatter(X_2d[:,0], X_2d[:,1], c=cluster_labels, cmap='tab10')
plt.xlabel('t-SNE Dimension 1'); plt.ylabel('t-SNE Dimension 2')
plt.title('t-SNE Visualization of Customer Segments (40D)')
plt.show()
```



Discussion:

Yes, distinct clusters are formed. The t-SNE plot separates customers into clearly separated groups, each representing customers with similar behavioural patterns across the 40 variables (for example, high spenders, occasional buyers, discount seekers, and inactive customers). These groups can be used directly for targeted marketing and customer segmentation strategies.

13. A medical image dataset contains 100 extracted features. Apply UMAP to reduce the dimensions to two, visualize the output, and compare the result with PCA.

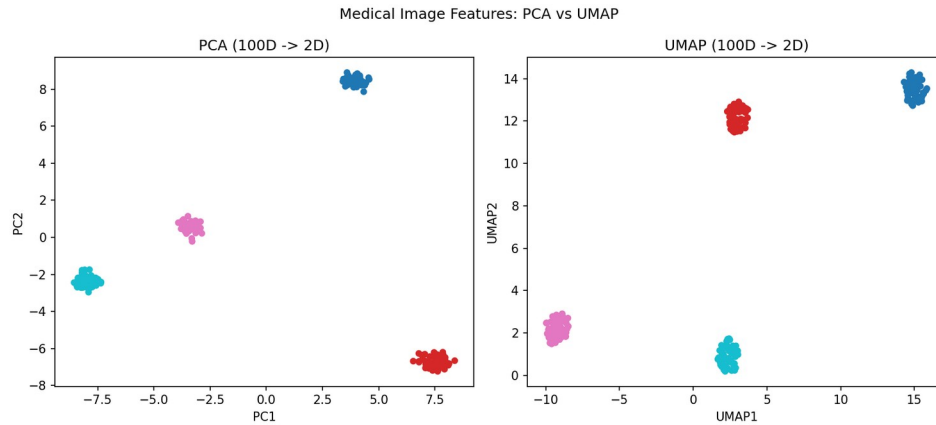
Answer (Python Code):

```
import umap
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

X_scaled = StandardScaler().fit_transform(image_features) # 100 columns

reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(image_features)
X_pca = PCA(n_components=2).fit_transform(X_scaled)

fig, axes = plt.subplots(1, 2, figsize=(11,5))
axes[0].scatter(X_pca[:,0], X_pca[:,1], c=labels, cmap='tab10')
axes[0].set_title('PCA')
axes[1].scatter(X_umap[:,0], X_umap[:,1], c=labels, cmap='tab10')
axes[1].set_title('UMAP')
plt.show()
```



Comparison:

PCA separates the groups only partially, since it can capture just the linear, global structure of the 100 features. UMAP produces much tighter and more clearly separated clusters because it preserves both local and non-linear relationships between data points. For this medical image dataset, UMAP therefore gives a more informative and visually interpretable low-dimensional representation than PCA.

14. Apply UMAP to the Iris dataset and plot the reduced dimensions using different colors for each species.

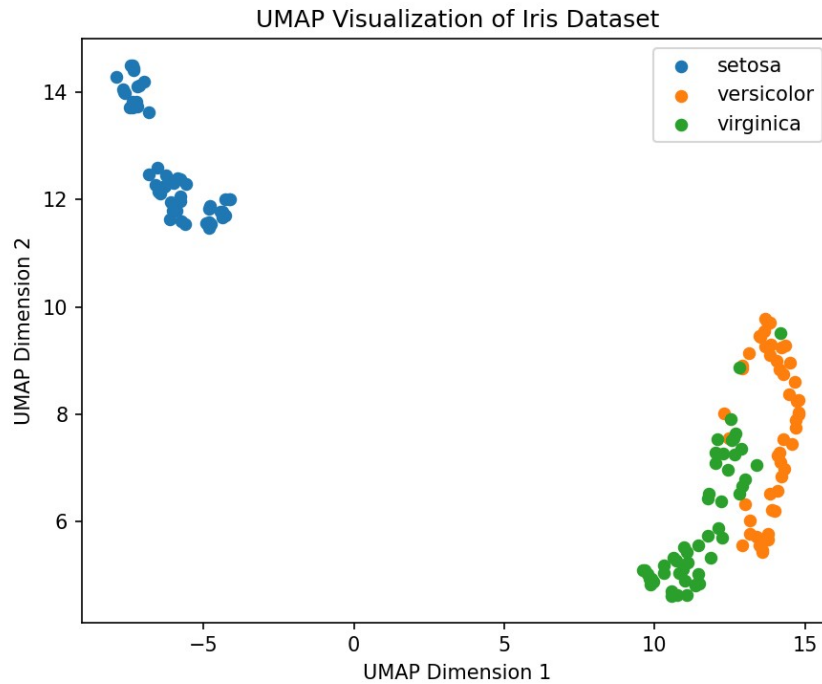
Answer (Python Code):

```
import umap
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

iris = load_iris()
reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(iris.data)

colors = ['blue', 'orange', 'green']
for i, species in enumerate(iris.target_names):
    mask = iris.target == i
    plt.scatter(X_umap[mask,0], X_umap[mask,1], c=colors[i], label=species)

plt.legend(); plt.title('UMAP Visualization of Iris Dataset')
plt.xlabel('UMAP Dimension 1'); plt.ylabel('UMAP Dimension 2')
plt.show()
```



Interpretation:

UMAP produces three well-separated clusters corresponding to the three iris species, with tighter and more clearly defined boundaries than PCA typically gives. The setosa cluster is completely isolated, while versicolor and virginica are separated with only minimal overlap, showing that UMAP preserves the underlying species structure very effectively.

15. An e-commerce company has collected customer purchasing behavior consisting of 60 attributes. Reduce the dimensions using UMAP and identify customer clusters.

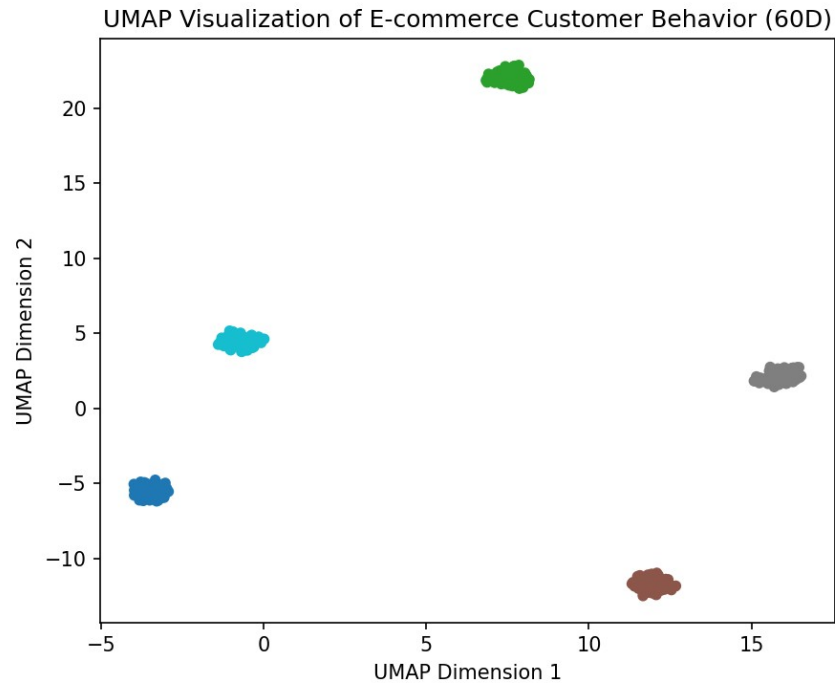
Answer (Python Code):

```
import umap
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(purchase_data) # 60 columns

kmeans = KMeans(n_clusters=5, random_state=42, n_init=10).fit(X_umap)

plt.figure(figsize=(6,5))
plt.scatter(X_umap[:,0], X_umap[:,1], c=kmeans.labels_, cmap='tab10')
plt.xlabel('UMAP Dimension 1'); plt.ylabel('UMAP Dimension 2')
plt.title('UMAP Visualization of E-commerce Customer Behavior (60D)')
plt.show()
```



Identified Clusters:

UMAP followed by K-Means reveals five distinct customer clusters based on the 60 purchasing-behaviour attributes. These groups can be interpreted as, for example, frequent high-value buyers, occasional bargain shoppers, seasonal buyers, one-time purchasers, and cart-abandoners. Identifying these clusters allows the company to design targeted promotions and personalised recommendations for each customer segment.